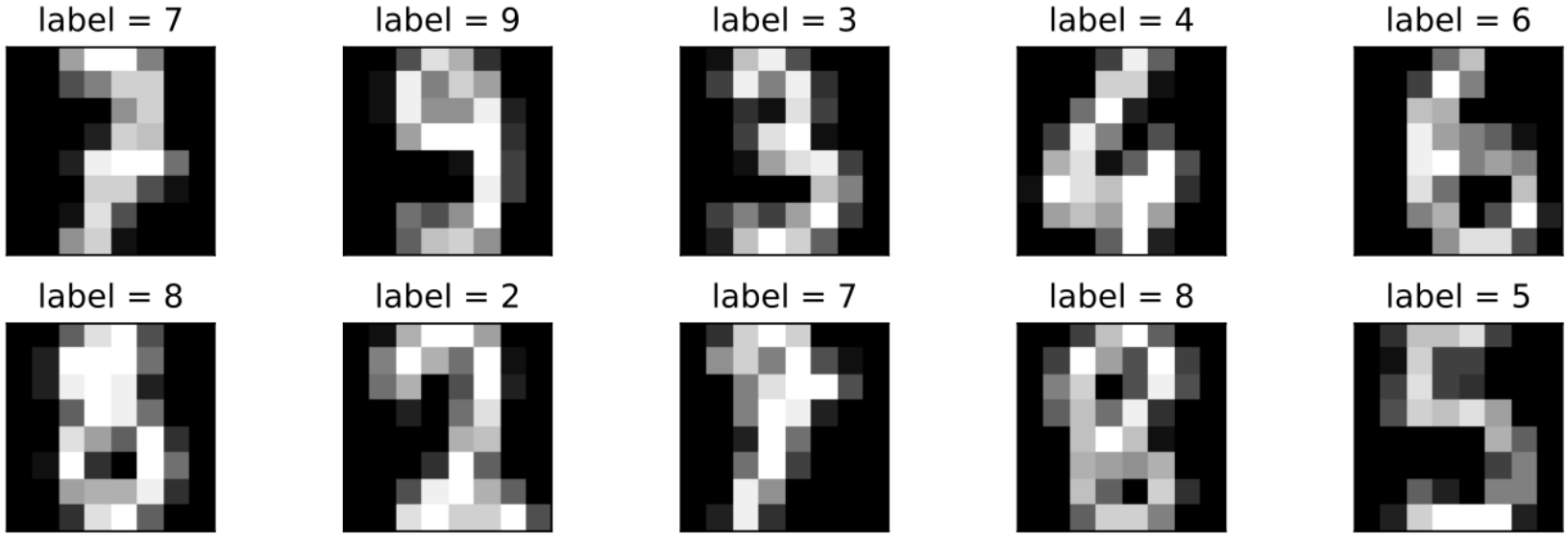
Classification of Handwritten Digits using Stochastic Gradient Descent

Álvaro Galisteo Bermúdez, Daniel Rath

16.02.2026

The Dataset

The dataset contains 1797 grayscale 8×8 images of handwritten digits. Each associated with a label.



It is available in python through the sklearn library

```
from sklearn.datasets import load_digits
```

Multiclass Logistic regression - I

We represent the label y_j of the image a_j as

$$p_j \in \mathbb{R}^{10}, \quad (p_j)_i = \begin{cases} 1 & \text{if } i = y_j \\ 0 & \text{otherwise} \end{cases}$$

The scores are given as

$$\varphi(a_j, x)_i = a_j^T w_i + b_i \qquad \text{with } x = (w_i, b_i)_{i=1}^{10} w_i \in \mathbb{R}^{64}, b_i \in \mathbb{R}$$

The prediction is given by

$$\tilde{y}_j = \underset{i \in \{0,\ldots,9\}}{\arg\max}\, \varphi(a_j; x)_i$$

We interpret the scores as probabilities by

$$\sigma(\varphi(a_j; x))_i = \frac{\exp(\varphi(a_j; x)_i)}{\sum_{l=0}^{9} \exp(\varphi(a_j; x)_l)}$$

Multiclass Logistic regression - II

The Optimization problem

The optimization problem we want to solve is

$$\underset{x \in (\mathbb{R}^{64} \times \mathbb{R})^{10}}{\text{minimize}} \frac{1}{N} \sum_{j=1}^{N} L(p_j, \varphi(a_j; x)),$$

where L is the cross-entropy loss defined as

$$L(p, s) = \log\Big(\sum_{l=0}^{9} \exp(s_l) \Big) - \sum_{i=0}^{9} p_i s_i$$

Cross entropy loss

$$L(p, s) = \log\Big(\sum_{l=0}^{9} \exp\left(s_l\right)\Big) - \sum_{i=0}^{9} p_i s_i$$

Using the cross-entropy loss ensures some important properties:

- Convexity of the optimization problem
- Differentiability for gradient descent methods
- Maximization of the likelihood of the observed data during training

Also note that we do not use the $\tilde{y} = \arg\max_{i \in \{0,\ldots,9\}} \ \varphi(a_j; x)_i$ in the loss function, because it is not differentiable.

L^2-Regularization

To prevent overfitting we add an L^2-regularization term to the optimization problem:

The Regularized Optimization problem

$$\underset{x \in (\mathbb{R}^{64} \times \mathbb{R})^{10}}{\text{minimize}} \frac{1}{N} \sum_{j=1}^{N} L(p_j, \varphi(a_j; x)) + \frac{\lambda}{2} \sum_{i=0}^{9} \|w_i\|^2$$

where $\lambda > 0$ is the regularization parameter. The regularization leads to

- smaller weights, which can help in generalization and prevent overfitting
- a unique finite optimum even if the data is linearly separable
- (numerically more stable optimization problem)

Stochastic Gradient Descent

For solving the optimization problem we use Stochastic Gradient Descent (SGD), i.e. we choose a mini-batch $\mathcal{B}_k$ of size B and calculate a stochastic estimator of the gradient as

$$\nabla L(x) = \frac{1}{B} \sum_{j \in \mathcal{B}_k} \nabla L(p_j, \varphi(a_j; x))$$

The update is then given by

$$x^{k+1} = x^k - \eta_k \nabla L(x^k)$$

This approach has some advantages and disadvantages, e.g.:

- Computationally efficient for larger datasets and small B
- Gradient can be to noisy if B is too small

SGD Implementation (Mini-batch) — I

Finite-sum objective (training set).

$$f_\lambda(W,b) = \frac{1}{N}\sum_{j=1}^{N} L\big(y_j,\ a_j^\top W + b\big) + \frac{\lambda}{2}\,\|W\|_F^2, \qquad W \in \mathbb{R}^{64\times 10},\ b \in \mathbb{R}^{10}.$$

Mini-batch gradient and update. For a mini-batch $\mathcal{B}_k$ of size B,

$$g_W^k = \frac{1}{B}\sum_{j\in\mathcal{B}_k} \nabla_W L_j(W^k, b^k) + \lambda W^k, \qquad g_b^k = \frac{1}{B}\sum_{j\in\mathcal{B}_k} \nabla_b L_j(W^k, b^k),$$

$$W^{k+1} = W^k - \eta\, g_W^k, \qquad b^{k+1} = b^k - \eta\, g_b^k.$$

Practical details.

- *Epoch shuffle:* permute training indices before forming mini-batches.
- *Reproducibility:* fix seeds for initialization and epoch shuffling.

SGD Implementation (Mini-batch) — II

Numerical stability (softmax). Given scores $S \in \mathbb{R}^{B\times 10}$, we compute

$$\text{softmax}(S)_{i\ell} = \frac{\exp\left(S_{i\ell} - m_i\right)}{\sum_{r=0}^{9} \exp\left(S_{ir} - m_i\right)}, \qquad m_i = \max_{0\le r\le 9} S_{ir},$$

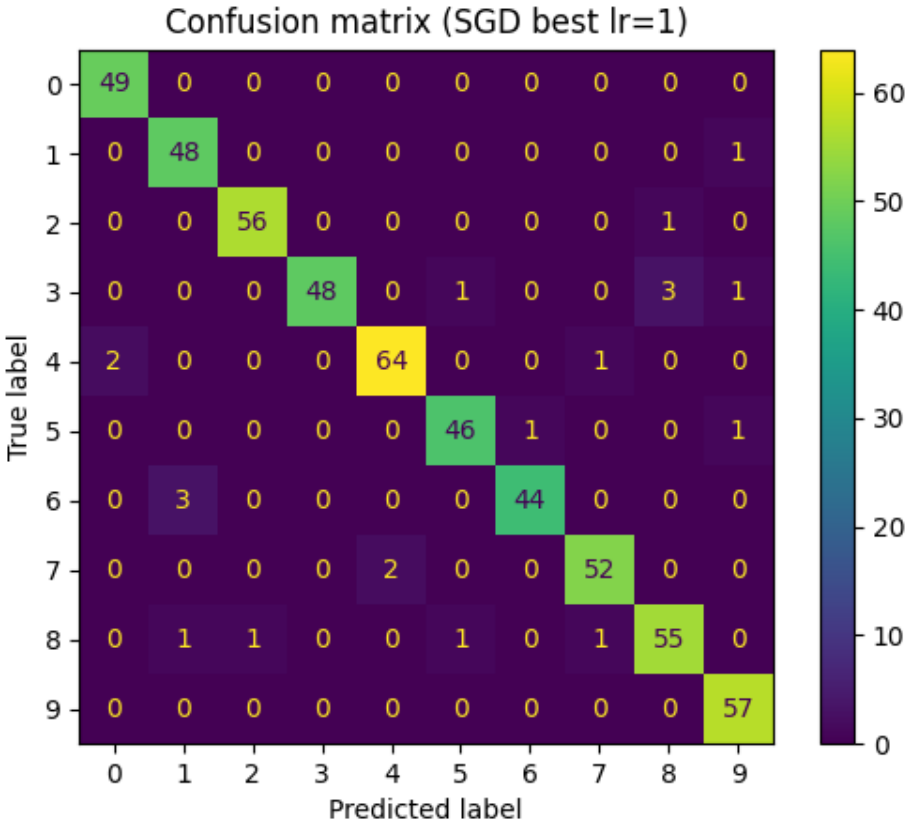
which avoids overflow in $\exp(\cdot)$.

Pseudo-code (one epoch).

```
perm = rng.permutation(N)
for start in range(0, N, B):
    idx = perm[start:start+B]
    dW, db = gradients(W, b, A[idx], y[idx],
        l2_reg=lambda_)
    W -= eta * dW
    b -= eta * db
```

Results with SGD

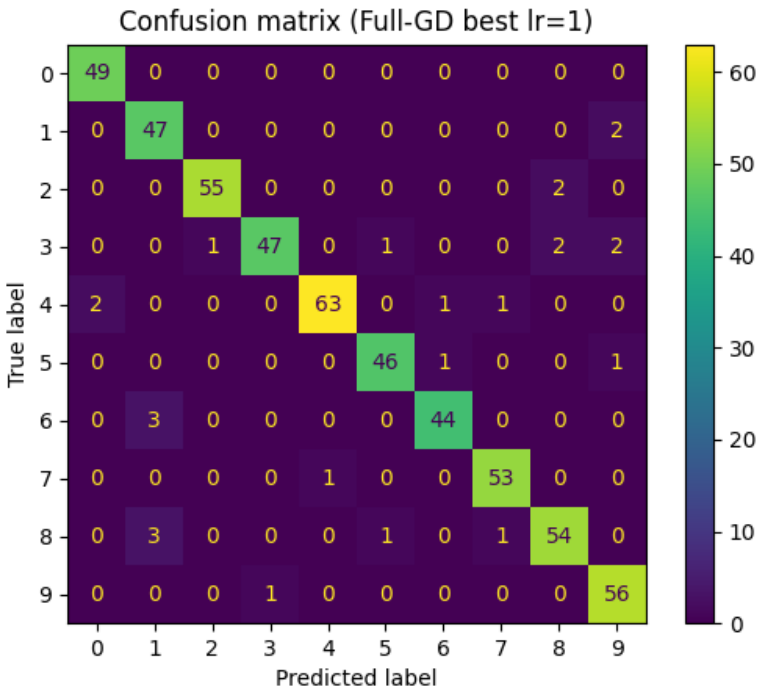
We trained the model for 50 epochs with a mini-batch size of $B = 64$ and a learning rate of $\eta_k = 1$ on a random training set of 70% of the data. The remaining 30% was used as a test set.



- Train accuracy: 0.983
- Test accuracy: 0.961
- Training Time: 30ms

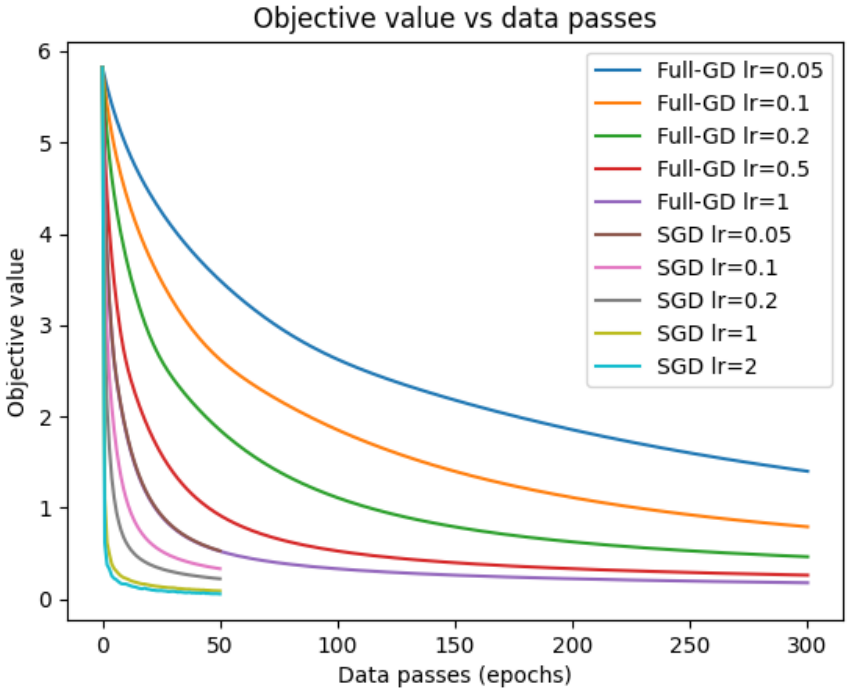
Results with FGD

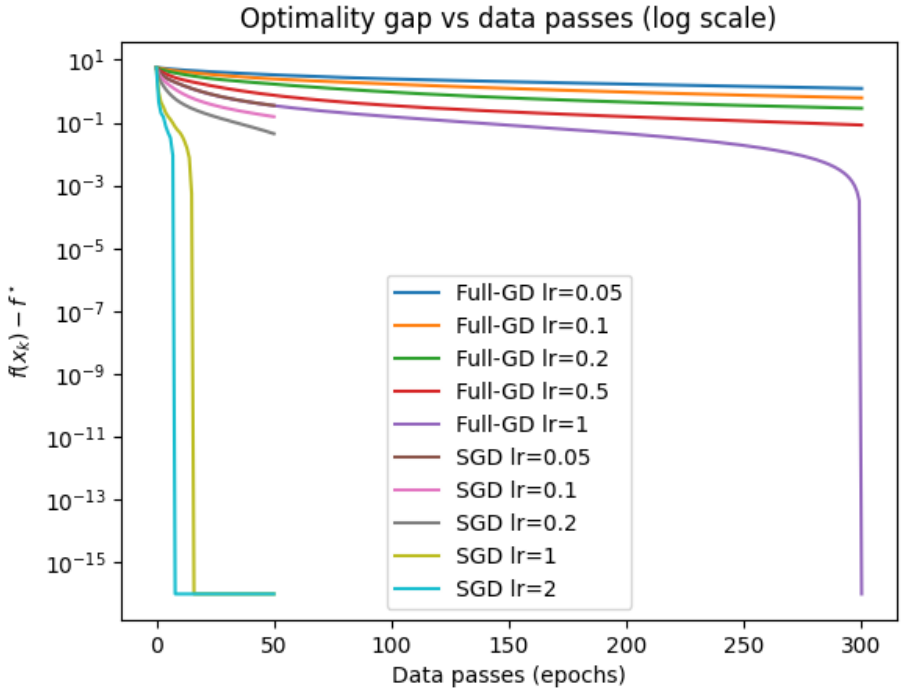
We trained a full gradient descent reference model for 300 iterations with a learning rate of $\eta_k = 1$ on a random training set of 70% of the data. The remaining 30% was used as a test set.



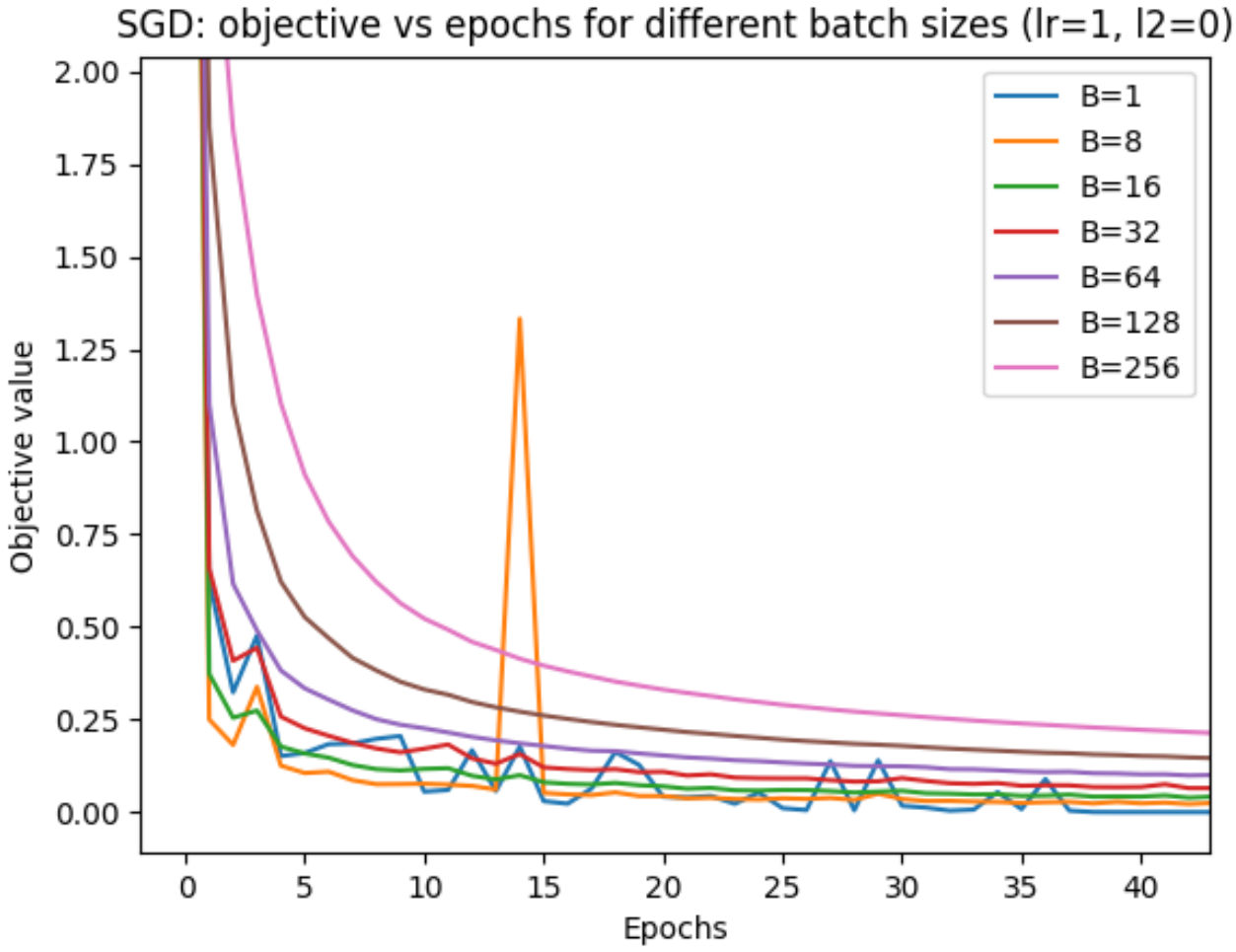
- Train accuracy: 0.957
- Test accuracy: 0.952
- Training Time: 90ms

Convergence Comparison



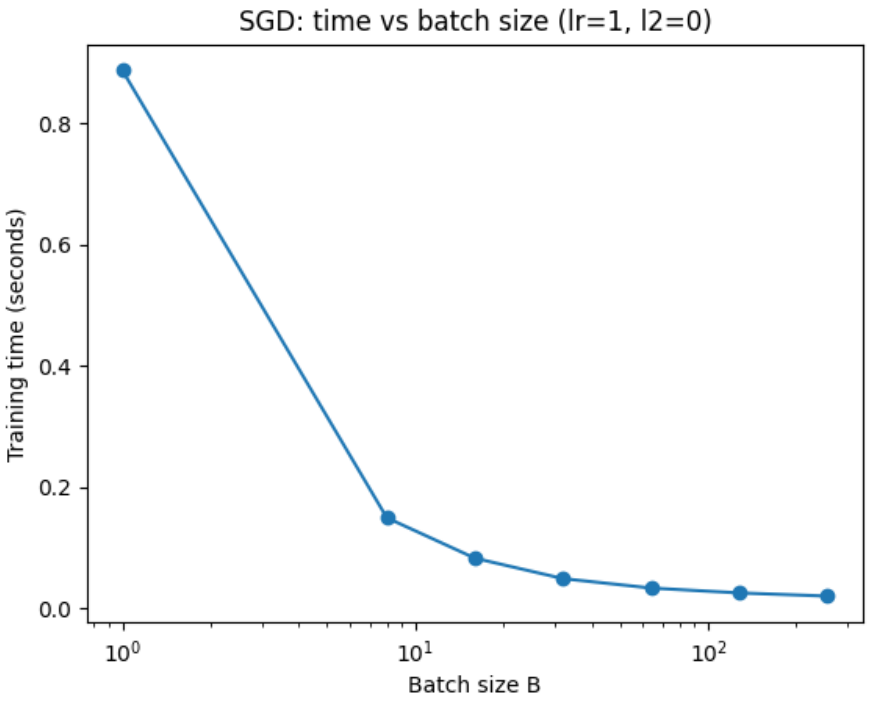


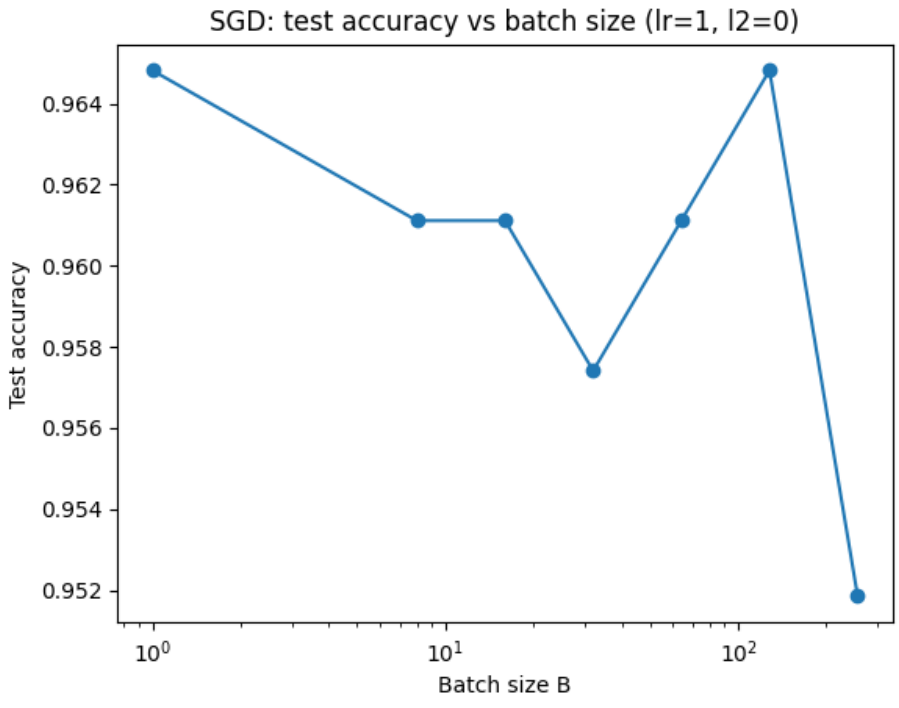
Different batch sizes - I



Result: Larger batch sizes lead to less noisy gradient estimates but also less frequent updates to the model.

Different batch sizes - II





Different L^2-regularization parameters